CRESTRON SIMPL#PRO PROGRAM LIBRARY

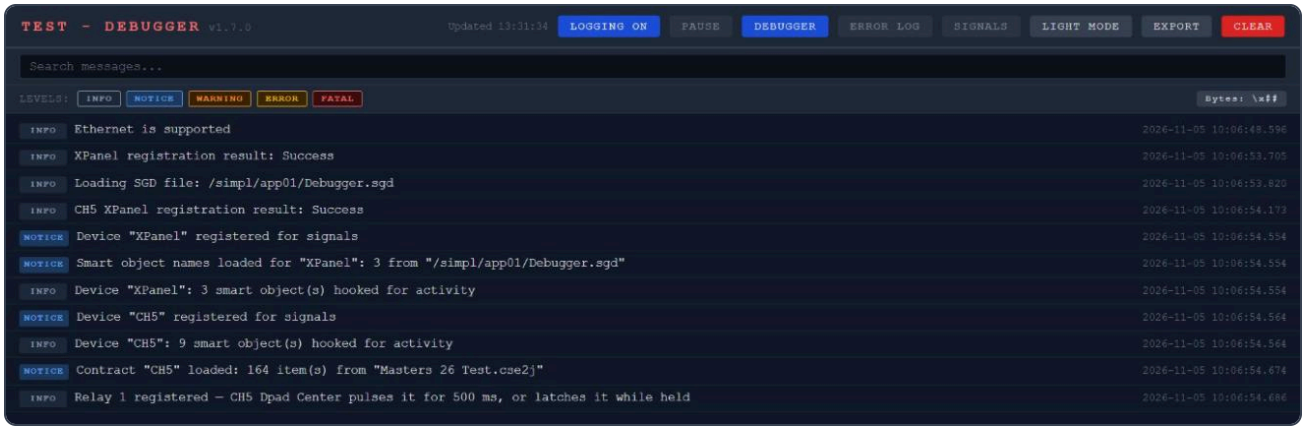
CwsDebug

Getting started — adding CwsDebug to a SIMPL#Pro program

Author	Mark Jervis
Company	Stage Front Presentation Systems
For version	1.7.0
Date	2026-11-05

What you'll end up with

A live, self-contained debug page hosted on the processor's own web server — no login, no extra services, nothing to deploy but the library itself. It streams your program's log messages as they happen, and can optionally show and drive a touchpanel's own signals the way Crestron Toolbox does. This walkthrough adds it to a program from a blank `InitializeSystem` in about ten minutes, then layers on the optional pieces.



The page you'll be looking at by the end of Step 4 — live search, severity filtering, a Bytes display switch, and light/dark themes, all built in.

> **API note:** the whole library is a *static* API. There is nothing to instantiate and no object to pass around — call `CwsDebugPage.Register(…)` and the various `CwsDebugPage.Log*(…)` methods directly, from anywhere, on any thread, even before `Register()` has run.

Before you start

You'll need:

- ✓ A SIMPL#Pro program targeting **.NET 8.0**.
- ✓ The `Crestron.SimplSharp.SDK.ProgramLibrary` NuGet package, version **2.21.257** or compatible — already a dependency of any SIMPL#Pro program.
- ✓ Either the `CwsDebug` project source (to reference directly) or a built `CwsDebug.dll` from a release.
- ✓ Network access from your workstation to the processor, to browse to the page once it's running.

Nothing else. The Signals panel steps later in this guide are optional and can be added at any time — they don't have to happen on the first pass.

Step-by-step

1 Add the reference

Reference the `CwsDebug` project (or the compiled DLL) from your SIMPL#Pro program's `.csproj`:

```
<ItemGroup>
  <ProjectReference Include="..\CwsDebug\CwsDebug.csproj" />
</ItemGroup>
```

If you're working from a built DLL instead of the source, use a plain `<Reference>` with a `<HintPath>` pointing at `CwsDebug.dll`, and copy the DLL alongside your program's own output.

2 Register the page

In `InitializeSystem()`, add one line:

```
using CwsDebug;

public class ControlSystem : CrestronControlSystem
{
    public override void InitializeSystem()
    {
        CwsDebugPage.Register("/debugger");
    }
}
```

`Register` takes an optional second argument for the ring buffer's size (default **2000** entries) and can be called with a different base path if your program hosts more than one page. Cleanup is automatic: the library releases the CWS server and flushes the log to disk when the program stops or restarts, so calling `Dispose()` / `Shutdown()` by hand is optional.

3 Start logging

Anywhere in your program — before or after `Register()`, on any thread — call one of the five severity methods:

```
CwsDebugPage.LogInfo("System starting");
CwsDebugPage.LogNotice("Loaded {0} devices", deviceCount);
CwsDebugPage.LogWarning("Low buffer on {0}", deviceName);
CwsDebugPage.LogError("Comms timeout on {0}", deviceName);
CwsDebugPage.LogFatal("Unrecoverable: {0}", reason);   // also mirrored to the
system ErrorLog

try { /* ... */ }
catch (Exception e)
{
    CwsDebugPage.LogException(e, "Error during init");   // full type + message +
stack trace
}
```

Each `Log*` method takes an optional `string.Format` template plus arguments, the same way `ErrorLog.Notice` does. Nothing needs to be wired up first — a call that arrives before `Register()` runs is still recorded and shown once the page comes up.

Messages show exactly as logged: runs of spaces, tabs and line breaks are kept, so aligned output stays aligned and the stack trace from `LogException` reads on several lines. Characters that would otherwise be invisible — control characters such as CR (`\x0D`), and bytes 0x80–0xFF — show as a tinted hex escape, so a device response can be read byte for byte. The **Bytes** button at the right of the level bar switches between `\x0D`, `[0D]` and raw.

4 Build, deploy, and browse to the page

Build and load the program the way you normally would. Once it's running, open the page from a browser on the same network:

Platform	URL
Appliance (RMC4, etc.)	`https://<processor>/cws/debugger/debugger.html`
VC-4 (Server)	`https://<server>/VirtualControl/Rooms/<roomId>/cws/debugger.html`

With the default base path on VC-4, the redundant segment collapses so the page serves directly at `.../cws/debugger.html`. If a program hosts more than one page, give each an explicit non-default base path such as `/example`; it stays in the URL on both platforms, for example `…/cws/example/debugger.html`. `<roomId>` is the room's ID in VC-4, not its display name. Use `http://` instead if the processor has HTTPS turned off.

That's the whole integration. Everything from here is optional.

5 Already logging with ErrorLog or CrestronConsole? OPTIONAL

Existing code doesn't need rewriting. Add two `using` aliases at the top of each file that logs, and every `ErrorLog.*` and `CrestronConsole.Print` / `PrintLine` call in that file goes to the debug page instead — the call sites stay exactly as they are:

```
using ErrorLog        = CwsDebug.CwsErrorLog;
using CrestronConsole = CwsDebug.CwsConsole;

ErrorLog.Notice("Loaded {0} devices", deviceCount);   // now shows on the page as
Notice
CrestronConsole.PrintLine("Relay {0} closed", n);     // one page entry per line
```

By default these write *only* to the page. To keep the old destinations as well, set `CwsErrorLog.MirrorToErrorLog = true` and/or `CwsConsole.PassThrough = true`. Console commands (`AddNewConsoleCommand` and the rest) still go straight to the real console. Console output is line-buffered: text printed without a newline appears once the line is finished, or within a few seconds if no newline arrives.

6 Add a touchpanel OPTIONAL

To browse and drive a touchpanel's own signals from the page — the Signals panel — register the device once, with any `BasicTriList` (XPanel, TSW, and other touchpanels):

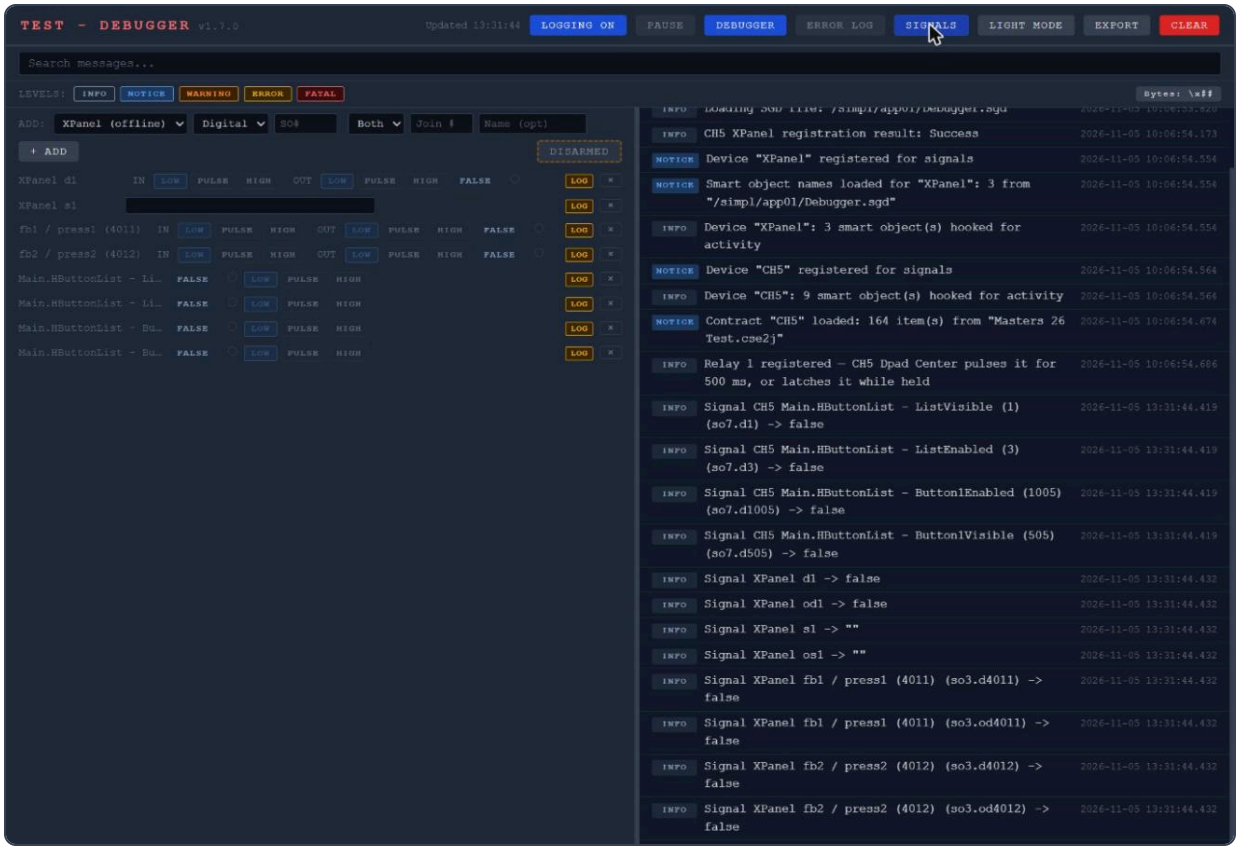
```
public override void InitializeSystem()
{
    CwsDebugPage.Register("/debugger");
    CwsDebugPage.RegisterDevice("Panel", tsPanel);
}
```

Call `UnregisterDevice` before disposing the device — a program's own stop handler usually runs before the library's, so the page can otherwise briefly reach a disposed panel:

```
CwsDebugPage.UnregisterDevice("Panel");
panel.Dispose();
```

Open **Signals** on the page and add a signal by device, type (Digital / Analog / Serial) and direction — **In** (into the panel: feedback, changes what it shows), **Out** (out of the panel: presses, changes what the system does), or **Both** together on one row. Digital signals are driven with **Pulse**, **High** and **Low**; analog and serial are edited by typing a value and pressing Enter. A digital **Both** row carries two labelled sets of buttons: **In** changes the feedback the panel shows, and **Out** acts like a press, running the program's own handlers. They are kept apart on purpose — lighting a button's feedback doesn't run the code behind it, and a press doesn't prove the feedback works.

Long names are clipped to fit the column; hover over one to see the full name, its device and its signal key. On a phone or narrow window the panel stacks above the log instead of beside it.



The Signals panel beside the log — digital Both rows with separate In and Out controls, a serial row, and smart-object rows named from the contract, each with its own Log toggle. Forcing stays off until the Disarmed button is pressed to arm it.

7 Label smart objects from the SGD OPTIONAL

Pass the same SGD path you already give `LoadSmartObjects`, and smart objects show up by their design-time names instead of bare numbers:

```
tsPanel.LoadSmartObjects(sgdPath);
CwsDebugPage.RegisterDevice("Panel", tsPanel, sgdPath);   // objects show as "Button List", etc.
```

Entering a **Smart Object ID** on the page then lists that object's own signals by name (`Item 1 Pressed`, `Select Item`, `Item 1 Text`) rather than by number, and the field offers the objects it knows about — type a number or part of a name to find one.

8 Register a Construct / CH5 contract OPTIONAL

For a Crestron Construct panel, register its contract instead, and signals can additionally be addressed by their contract item path — smart objects and signals then list by contract name (`Main.Dpad` with `Up`, `Down`, `Left`, `Right`, `Center`) instead of by number:

```
CwsDebugPage.RegisterContract("Touchpanel", ch5Panel, "My Project.cse2j");
```

The manifest is the `<Project>.cse2j` file from the Construct project's `output\contract\mapping\` folder; ship it as content in your program's `.csproj` (a `CopyToOutputDirectory` entry) so it exists on the processor. If it's missing or unreadable the panel still registers and everything remains reachable by number — a warning in the log says so, rather than failing quietly.

> On an **Out** row for a contract-backed signal, the same Pulse / High / Low buttons make the program react as though the panel had actually been used — its own signal handlers and Construct contract events run normally. That means the code behind a button on the panel can be exercised straight from the page, with nothing added to the program. Where that isn't available for a given signal, the buttons simply don't appear and the row stays a live read-out.

9 Turn on per-signal logging OPTIONAL

Every row in the Signals panel has its own **Log** toggle (on by default for a newly added row). With it on, the processor writes a line to the log every time that signal changes — no code required:

```
Signal Panel Power Button (od1) -> true
Signal Panel Volume (a1) -> 46536
```

Lines come from the processor itself, so they carry the exact moment the change happened and catch transitions too brief to land on a poll. Digital edges are always logged; analog and serial have a floor between entries so a dragged slider can't flood the buffer. Logging a signal stops shortly after the page stops asking for it, so closing the browser can't leave a processor logging indefinitely.

Putting it all together

A minimal program using every piece from this guide:

```
using CwsDebug;

public class ControlSystem : CrestronControlSystem
{
    private XpanelForSmartGraphics _panel;

    public override void InitializeSystem()
    {
        try
        {
            CwsDebugPage.Register("/debugger");
            CwsDebugPage.LogInfo("System starting");

            _panel = new XpanelForSmartGraphics(0x03, this);
            _panel.LoadSmartObjects(@"\NVRAM\Panel.sgd");
            _panel.Register();
            CwsDebugPage.RegisterDevice("Panel", _panel, @"\NVRAM\Panel.sgd");

            CwsDebugPage.LogNotice("Panel registered");
        }
        catch (Exception e)
        {
            CwsDebugPage.LogException(e, "Error during init");
        }
    }

    public override void ControllerProgramEventHandler(eProgramStatusEventType type)
    {
        if (type == eProgramStatusEventType.Stopping)
        {
            CwsDebugPage.UnregisterDevice("Panel");
            _panel?.Dispose();
        }
    }
}
```

Keeping it safe on a live network

The debug page has no login — anyone who can reach the processor on the network can open it, read the log, clear it, or (if a device is registered) change signals. On a dedicated control-system network this is normally fine; on a network shared with general office traffic, avoid logging anything sensitive and ask your IT administrator about a restricted VLAN.

Signal changes are switched off by default: forcing needs to be armed on the page, arming is forgotten on every restart and after five idle minutes, and every change is written to the log with the

address it came from. For a system being handed over, set `CwsDebugPage.ValueWritesAllowed = false` to keep the panel view-only while leaving the log running.

Where to go next

- The full API reference — every member, property, and platform note — is in `CwsDebug_Documentation.pdf`, alongside this guide.
- Release history and what changed in each version is in `CHANGELOG.md`.
- Source and releases: github.com/markjervis/CwsDebug.